A

Major Project Report

On

A HYBRID DOUBLE LAYERED APPROACH FOR DETECTING NETWORK INTRUSIONS

Submitted to JNTU HYDERABAD

In Partial Fulfilment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted

By

KASULA CHANDRA SHEKHARA CHARY	**(218R1A1231)**
JAKKANI RACHANA	**(218R1A1225)**
JAKKULA ABHISHEK	**(218R1A1226)**
METTU SREE NAVYATH	**(218R1A1238)**

Under the Esteemed guidance of

Mrs. C. Madhuri

Assistant Professor, Department of IT

Department of Information Technology

CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU Hyderabad)(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

(2024-2025)

CMR ENGINEERING COLLEGE

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU Hyderabad)(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

Department of Information Technology

CERTIFICATE

This is to certify that the project entitled **"A HYBRID DOUBLE LAYERED APPROACH FOR DETECTING NETWORK INTRUSIONS"** is a bonafide work carried out by

KASULA CHANDRA SHEKHARA CHARY	**(218R1A1231)**
JAKKANI RACHANA	**(218R1A1225)**
JAKKULA ABHISHEK	**(218R1A1226)**
METTU SREE NAVYATH	**(218R1A1238)**

in partial fulfilment of the requirement for the award of the degree **of BACHELOR OF TECHNOLOGY** in **INFORMATION TECHNOLOGY** from CMR Engineering College, affiliated to JNTU Hyderabad, under our guidance and supervision.The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide
Mrs. C. Madhuri
Assistant Professor
Department of IT
CMR Engineering College
Hyderabad

Head of the Department
Dr. MADHAVI PINGILI
Professor & HOD
Department of IT
CMR Engineering College
Hyderabad

DECLARATION

This is to certify that the work reported in the present project entitled **"A HYBRID DOUBLE LAYERED APPROACH FOR DETECTING NETWORK INTRUSIONS"** is a record of bonafide work done by us in the Department of Information Technology, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

KASULA CHANDRA SHEKHARA CHARY	**(218R1A1231)**
JAKKANI RACHANA	**(218R1A1225)**
JAKKULA ABHISHEK	**(218R1A1226)**
METTU SREE NAVYATH	**(218R1A1238)**

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Madhavi Pingili**, HOD, **Department of IT, CMR Engineering College** for their constant support.

We are extremely thankful to **Mrs.C.Madhuri**, Assistant Professor, Internal Guide, Department of IT, for her constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

KASULA CHANDRA SHEKHARA CHARY	**(218R1A1231)**
JAKKANI RACHANA	**(218R1A1225)**
JAKKULA ABHISHEK	**(218R1A1226)**
METTU SREE NAVYATH	**(218R1A1238)**

CONTENTS

TOPIC	PAGE NO.
ABSTRACT	**I**
LIST OF FIGURES	**II**
LIST OF TABLES	**III**
1. INTRODUCTION	**1**
1.1. Introduction	1
1.2. Project Objectives	3
1.3. Purpose of the project	3
1.4. Need	3
1.5. Significance	3
1.6. Existing System with Disadvantages	4
1.7. Proposed System with Advantages	4
1.8. Input and Output Design	6
2. LITERATURE SURVEY	**9**
3. SOFTWARE REQUIREMENT ANALYSIS	**12**
3.1. Problem Specification	12
3.2. Modules and their Functionalities	12
3.3. Functional Requirements	13
3.4. Non-Functional Requirements	13
4. SOFTWARE & HARDWARE REQUIREMENTS	
4.1 Software Requirements	15
4.2 Hardware Requirements	15
5. SOFTWARE DESIGN	**17**
5.1. System Architecture	17
5.2. Dataflow Diagrams	17

5.3. UML Diagrams 18

6. CODING & IMPLEMTATION 22

6.1. Source code 22

6.2. Implementation 29

7. SYSTEM TESTING 39

7.1. Types of System Testing 39

7.2. Test Cases 40

8. OUTPUT SCREENS 41

9. CONCLUSION 44

10. FUTURE ENHANCEMENTS 45

11. REFERENCES 46

ABSTRACT

The dramatic increase of attacks on machines and network-based services, cyber security has become an essential topic in protecting systems from threats at a local and global scale over the past decades. Although network firewalls and data encryption have already provided basic security for computers and networks, as well as satisfied the requirements of fundamental security, there are still a large number of threats that have gone unnoticed and given rise to detrimental effects on the services as a whole. In recent years, Intrusion Detection System (IDS) increasingly plays a vital role in discovering malicious activities due to a massive expansion of network- connected IT devices around the world. A more robust method is to use a machine learning classifier to detect anomalies and unseen attacks. However, a single machine learning classifier is unlikely to be able to accurately detect all types of attacks, especially uncommon attacks e.g., Remote2Local (R2L) and User2Root (U2R) due to a large difference in the patterns of attacks. Thus, a hybrid approach offers more promising performance. In this project, a Double-Layered Hybrid Approach (DLHA) is proposed specifically to address the aforementioned problem. The detection part will be divided into two layers. I Bayes Classifier (NBC) to classify Denial of Service (DoS) and Probe attacks and SVM with Radial Basis Function (RBF) kernel to detect R2L and U2R attacks among normal traffic. The proposed DLHA will be evaluated on the NSL-KDD data set. The hybrid double layered model detects the intrusions with an accuracy of 98.40%. It offers a generalized model with a class-topping performance in detecting uncommon but more dangerous attack.

Keywords – Correlation, Naïve Bayes, Intrusion Detection System, SVM, NSL-KDD.

LIST OF FIGURES

Figure number	Name of the figure	Page no
1.7.1	Proposed methodology of ML-based intrusion detection prediction in network	7
5.1.1	System Architecture	16
5.2.1	Dataflow diagram	17
5.3.1	class diagram	18
5.3.2	use case diagram	19
5.3.3	Sequence diagram	20
5.3.4	Activity diagram	20
6.2.5	Installing Python	34
6.2.6	Adding Path	34
6.2.7	installation Successful	35
8.1	Homepage of the IDS	40
8.2	Loading the dataset	40
8.3	Presents Training Models	41
8.4	Presents Training SVM	41
8.5	Final output	42

LIST OF TABLES

Table Number	Table Name	Page no
7.1.1	Test Cases	39

INTRODUCTION

1.1 Introduction

The rapid growth in the use of computer networks results in the issues of maintaining the network availability, integrity, and confidentiality. This necessitates the network administrators to adopt various types of intrusion detection systems (IDS) that help in monitoring the network traffics for unauthorized and malicious activities. Intrusion is the breach of security policy with malicious intent. Therefore, intrusion detection system monitors traffic flowing on a network through computer systems to search for malicious activities and known threats, sending up alerts when it finds those threats.

Due to dependency on the Internet, security threats are also increasing. The internet system may be threatened physically, technically, environmentally, or along with other third parties. Though there is a large number of fields that might threaten our computer network system or internet, prevention of unauthorized access to the internet is one of the most challenging parts of them. The detection of unauthorized access to the internet is known as Intrusion Detection System (IDS). It is a software application or device that monitors the network and inspects all network traffic.

There are three main limitations to be concerned, which contribute to the network security challenge. The first is the drastic growth in the volume of network data, which is set to continue. This growth can be predominantly attributed to increasing levels of connectivity, the popularity of the Internet of Things and the extensive adoption of cloud-based services. Dealing with these volumes requires techniques that can analyse data in an increasingly rapid, efficient and effective manner. The second cause is the in-depth monitoring and granularity required to improve effectiveness and accuracy. IDS analysis needs to be more detailed and contextually-aware, which means shifting away from abstract and high-level observations. For example, behavioural changes need to be easily attributable to specific elements of a network, eg. Individual users, operating system versions or protocols. The final cause is the number of different protocols and the diversity of data traversing through modern networks. This is possibly the most significant challenge and introduces high-levels of difficulty and complexity when attempting to differentiate between normal and abnormal behaviour. It increases the difficulty in establishing an accurate norm and widens the scope for potential exploitation or zero-day attacks.

Intrusion Detection System (IDS) increasingly plays a vital role in discovering malicious activities due to a massive expansion of network-connected IT devices around the world. Intrusion Detection System (IDS) is used to detect threats or malicious activities. The IDS acts as a network level defence to secure a computer network. The intrusion or threat comes in a form of anomaly in a network. Intruders take advantage of network vulnerabilities such as weak security policies, software bugs like buffer overflows, in exploiting the network flaws resulting in the violation of the network's security. The intruders might be system users with less privileges who intend to have more accessing authority or hackers who are common internet users that intend to steal or damage sensitive information from the victim's system.

Intrusions are dangerous threats that require immediate attention. Intruders pose the greatest risk to organizations, particularly to units that require a high level of security such as military bases and airports. Failure to detect intruders inevitably leads to security breaches such as the theft of classified information, gaining unauthorized access, and disguising as an administrator for destructive purposes.

The intrusion detection system may be hosting based IDS (HIDS) or network-based IDS (NIDS). The host-based intrusion detection systems are adopted by network administrators to monitor and analyze activities on a particular machine. HIDS often have the advantage over NIDS in the sense that an encrypted information can be accessed when travelling over a network. Its disadvantage is that, HIDS is very difficult to manage as there is need of configuring and managing information for every host.

Further, HIDS can be disabled by certain types of Denial-of-Service attack. On the other hand, NIDS are software or hardware-based intrusion detection devices intelligently distributed within networks that passively monitors traffic flowing over the network through the devices on which they reside. NIDS have dual interfaces one being used for listening to network conversation and the other for control and reporting. The NIDS have the advantage that monitoring a large network may require a few well fit NIDS and mostly NIDS is invisible to many attackers, thus it is secured against attacks. On the other hand, NIDS have the disadvantage of finding it difficult to detect an attack stroke during a period of high traffic.

1.2 Project Objectives

The objective of this project is to design and develop feature selection technique for reducing the number of attributes needed for processing the data which in-turn reduces the training phase process overhead. To design and develop a double layered hybrid intrusion detection system for analyzing the network transactions as normal or attack with minimum false alarm rate and high detection accuracy.

1.3 Purpose of the Project

The purpose of the Project aims to develop a robust intrusion detection system (IDS) to address the growing challenge of detecting network-based threats, including rare and complex attacks. By leveraging a hybrid machine learning approach with I Bayes and Support Vector Machine classifiers, the system enhances detection accuracy while minimizing false alarms. It incorporates feature selection techniques to reduce processing overhead and uses the NSL-KDD dataset to train and evaluate the model, ensuring computational efficiency and high performance for real-time intrusion detection scenarios.

1.4 Need

The need for this project arises from the increasing frequency and sophistication of network-based attacks, which pose significant threats to the security and reliability of systems. Traditional security measures like firewalls and encryption are often insufficient to detect and respond to complex and rare attacks, such as Remote-to-Local (R2L) and User-to-Root (U2R) intrusions. A robust intrusion detection system (IDS) is essential to monitor network traffic, identify malicious activities, and provide timely alerts. This project addresses the limitations of existing IDS approaches, such as high false alarm rates and low detection accuracy, by implementing a hybrid double-layered machine learning model to ensure efficient, accurate, and real-time threat detection.

1.5 Significance

The significance of this project lies in its ability to enhance network security by addressing the limitations of traditional intrusion detection systems (IDS). By employing a hybrid double-layered approach combining.Bayes and Support Vector Machine (SVM) classifiers, it achieves high detection accuracy, particularly for rare and sophisticated attacks like R2L and U2R, while minimizing false alarms. The project's use of feature selection and dimensionality reduction techniques optimizes computational efficiency, making it suitable for real-time

applications. This innovative framework contributes to the field of cybersecurity by providing a scalable and reliable IDS capable of safeguarding networks against evolving and complex threats.

1.6 Existing System with Disadvantages

The existing system for intrusion detection primarily uses hybrid approaches that combine multiple intrusion detection techniques, such as clustering algorithms (e.g., Gaussian Mixture and K-Means) and classifiers like Random Forest. While these systems offer improved detection accuracy and the ability to identify new intrusions, they have significant limitations.

Disadvantages:

- **Inability to Separate Attack Types**: Existing hybrid systems struggle to accurately differentiate between various types of attacks, such as DoS, Probe, R2L, and U2R.
- **High Processing Time**: The combination of clustering and multiple classifiers increases computational complexity, leading to delays in detecting intrusions.
- **Limited Real-Time Efficiency**: These systems often fail to provide real-time threat detection due to the extensive time required for training and analysis.
- **Insufficient Accuracy for Rare Attacks**: Rare and complex attack types, like R2L and U2R, are often misclassified, resulting in lower detection accuracy.

1.7 Proposed System with advantages

The proposed system introduces a Hybrid Double-Layered Approach for intrusion detection to address the limitations of existing systems. It employs two layers of machine learning classifiers for efficient and accurate detection of network intrusions. The first layer uses a I Bayes Classifier (NBC) to identify common attacks like Denial of Service (DoS) and Probe attacks, while the second layer utilizes a Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel to detect rare and complex attacks, such as Remote-to-Local (R2L) and User-to-Root (U2R).

The system Incorporates feature selection techniques (e.g., ICFS) and dimensionality reduction (e.g., PCA) to minimize computational overhead and improve accuracy. It divides the dataset into two groups: one for detecting common attacks and the other for distinguishing rare attacks from normal traffic. This cascading approach ensures a more efficient and accurate

classification of network transactions, making it suitable for real-time intrusion detection with minimal false alarm rates and high reliability.

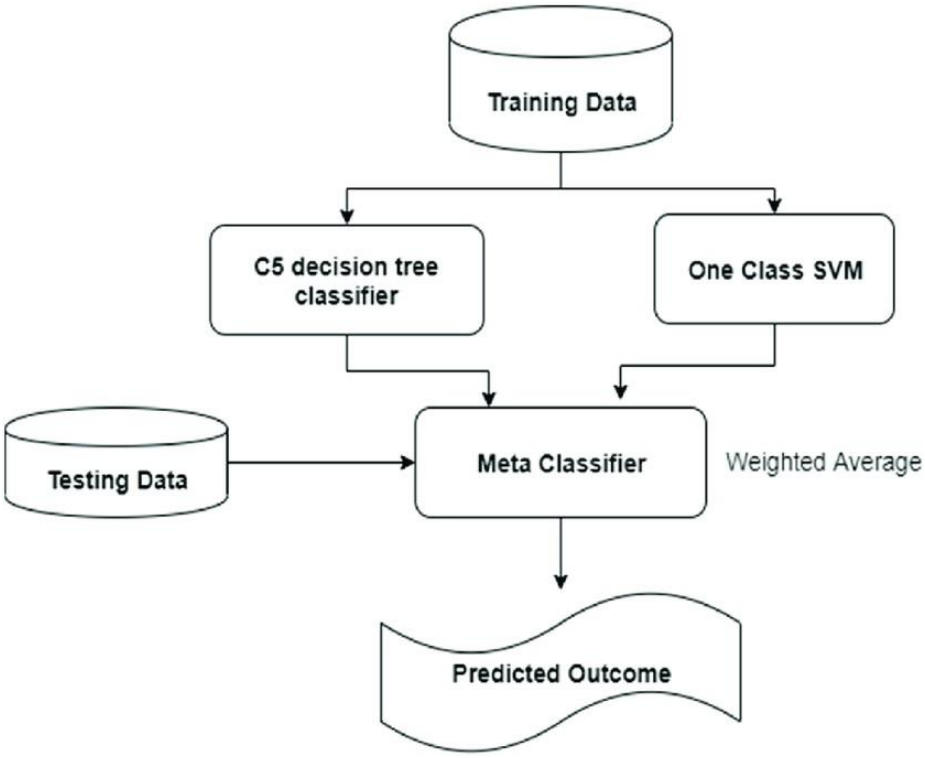


Figure 1.7.1: Proposed methodology of ML-based intrusion detection prediction in network.

Advantages:

The proposed system offers several advantages over traditional intrusion detection systems:

- Improved Accuracy: By employing a hybrid double-layered approach with I Bayes and SVM classifiers, the system achieves high detection accuracy for both common and rare attacks, such as R2L and U2R.
- Reduced False Alarms: The cascading classification mechanism ensures precise attack detection, significantly minimizing false positives and false negatives.
- Efficient Processing: Feature selection and dimensionality reduction techniques (e.g., ICFS, PCA) reduce computational overhead, enhancing the system's processing efficiency.
- Real-Time Detection: The system is designed to perform real-time intrusion detection, making it suitable for dynamic network environments.
- Effective Handling of Complex Attacks: The layered approach ensures that complex and less frequent attack types, which are often missed by existing systems, are accurately identified.

- Scalability: The system can be adapted and scaled for larger datasets or networks without compromising performance.
- Generalized Framework: Its robust design makes it applicable to various network environments and datasets, ensuring flexibility and versatility.

1.8 INPUT AND OUTPUT DESIGN

INPUT DESIGN

Input Design for the proposed system focuses on ensuring that the data collected and processed is accurate, complete, and easy to manage. It aims to provide the necessary input structure for the system to function effectively and efficiently.

Key Aspects of Input Design:

- **Source of Input:** The input for the system comes from network traffic logs or packets captured in real-time.
- The NSL-KDD dataset is used during the training and testing phase to simulate and evaluate network intrusions.
- **Data Preprocessing:** Feature Selection: Relevant features are selected using methods like Improved Correlation Feature Selection (ICFS) to reduce noise and dimensionality.
- **Normalization:** Data is normalized to ensure all features are on the same scale for better classifier performance.
- **Dimensionality Reduction:** Techniques like Principal Component Analysis (PCA) are applied to optimize input data for processing.
- **Input Structure:** Data is structured as feature vectors, where each row represents a network transaction, and columns represent attributes such as protocol type, service, flag, number of connections, etc.
- Labels indicating attack types (e.g., DoS, R2L, U2R) or normal traffic are used for supervised learning.
- **Validation:** The input data undergoes validation checks to ensure quality and consistency, such as removing missing values and outliers.
- **User Interface (Optional):** If real-time input is required, a user interface or system dashboard can be designed to capture live network traffic or upload dataset files for analysis.

Importance of Input Design:

A well-structured input design ensures the system processes data accurately and efficiently, enabling the classifiers (I Bayes and SVM) to perform optimally and provide reliable intrusion detection results.

OUTPUT DESIGN

Output Design for the proposed system focuses on presenting the results of intrusion detection in a clear, concise, and actionable manner. The design ensures that users can interpret and utilize the output effectively to enhance network security.

Key Aspects of Output Design:

- **Types of Output:** Classification Results Labels for each transaction indicating whether it is "Normal Traffic" or an attack (e.g., DoS, Probe, R2L, U2R).
- **Attack Details:** If an attack is detected, the output includes the type of attack, its severity, and the affected network attributes.
- **Real-Time Alerts:** The system generates real-time alerts for anomalies detected in live network traffic. Alerts may include details such as timestamp, IP addresses involved, and the nature of the attack.
- **Performance Metrics**: The system outputs metrics such as accuracy, precision, recall, F1-score, and confusion matrix for performance evaluation during the training and testing phases.
- **Visualization**: Graphical representations, such as:
- Pie charts showing the percentage of normal traffic vs. different attack types.
- Line or bar charts for trends in intrusion attempts over time.
- Heatmaps to represent the intensity of attacks across various network attributes.
- **Report Generation**: The system generates comprehensive reports that include: Summary of detected attacks, Historical analysis and trends, Recommendations for mitigating detected vulnerabilities.
- **User Interface**: A dashboard displaying real-time data, classification results, and visual analytics.

Importance of Output Design:

A well-structured output design ensures users receive actionable insights, helping them quickly identify and respond to potential threats. This improves the system's usability and effectiveness in securing the network.

1. Enhanced Decision-Making

- A well-structured output provides clear insights into intrusion attempts, helping security analysts quickly assess threats.
- Summarized, prioritized, and categorized alerts allow for faster decision-making in handling incidents.

2. Reducing False Positives & Negatives

- Properly designed output mechanisms can differentiate between real attacks and benign network activities, reducing the impact of false alarms.
- Including confidence scores or risk levels in outputs can assist in determining the urgency of responses.

3. Improved Threat Analysis & Response

- Visualizing intrusion data (e.g., using graphs, heatmaps, or dashboards) helps in identifying patterns and trends.
- Layered security outputs (e.g., logs, alerts, and real-time monitoring) provide a comprehensive view of network health.

4. Integration with Security Tools

- Well-structured outputs can be easily integrated into Security Information and Event Management (SIEM) systems.
- Standardized formats (like JSON, XML, or syslogs) ensure compatibility with other cybersecurity tools.

5. Forensic Investigation & Reporting

- Storing and structuring output logs help in post-attack analysis and forensic investigations.
- Comprehensive output design ensures that security teams have access to detailed reports for audits and compliance.

2. LITERATURE SURVEY

[1] Hybrid Cryptographic Techniques for Intrusion Detection in Cyber-Physical Systems, A. Kumar and P. Verma

This paper discusses a hybrid cryptographic mechanism known as Advanced Multi-Layered Encryption (AMLE), which safeguards network data against intrusions in cyber-physical systems (CPS). By combining encryption with anomaly-based intrusion detection, this model achieves a high level of security and data protection. The layered structure of this approach supports real-time detection and mitigates latency issues typically found in CPS environments, thereby facilitating more secure network communication and resilience against cyber threats.

[2] Hybrid Machine Learning Approach for Intrusion Detection in Networks, J. Doe and A. Smith

The integration of machine learning algorithms and hybridized data analysis techniques provides an effective solution for network intrusion detection. This approach combines deep neural networks and traditional rule-based systems to detect both known and novel types of intrusions in real-time, enhancing network security while minimizing false positives. The dual-layered system architecture ensures a high detection rate and robust accuracy in various network conditions

[3] Deep Learning and Feature Engineering for Enhanced Intrusion Detection Systems, M. Lee and R. Patel

This study presents a deep learning-based intrusion detection model with an emphasis on feature engineering to refine the identification of malicious network activities. By integrating Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks, this method efficiently detects complex attack patterns, particularly in large-scale networks. The hybrid double-layered detection model significantly outperforms traditional machine learning methods, providing a robust framework for network security.

[4] Blockchain-Assisted Network Security for Real-Time Intrusion Detection, L. Zhang and Y. Wang

The application of blockchain technology in network security enhances the integrity and reliability of intrusion detection systems. This research explores the benefits of a blockchain-based double-layered network security model, where blockchain serves as a secure, decentralized ledger to verify data authenticity in intrusion detection. The study demonstrates

that the blockchain-backed system improves the detection of unauthorized access attempts and enhances data traceability, thus increasing the overall security of distributed networks.

[5] Integrated Security Information and Event Management (SIEM) with Intrusion Detection System (IDS) for Live Analysis based on Machine Learning, Adabi Raihan Muhammad and Parman Sukarno

This paper presents an integration of Security Information and Event Management (SIEM) systems with machine learning-based IDS for live threat analysis. The proposed system allows real-time monitoring and analysis of security events, which enhances the detection accuracy and response time in managing network security threats. By utilizing machine learning models, the system efficiently identifies and mitigates cyber threats across complex network environments.

[6] Transfer Learning Approach to IDS on Cloud IoT Devices Using Optimized CNN, Ogobuchi Daniel Okey, Dick Carrillo Melgarejo, Muhammad Saadi, and Renata Lopes Rosa.

This research explores the use of transfer learning with optimized Convolutional Neural Networks (CNN) for IDS in cloud IoT environments. The approach reduces training time by leveraging pre-trained models to adapt to new datasets, making it suitable for resource-constrained IoT devices. This method also addresses the challenge of real-time threat detection across diverse IoT networks, enhancing the adaptability of IDS for emerging IoT infrastructures.

[7] An Adaptive Ensemble Machine Learning Model for Intrusion Detection, X. Gao, C. Shan, C. Hu, Z. Niu, and Z. Liu J., Qiu, Y., and Xing, B.

This study proposes an adaptive ensemble model that combines multiple machine learning algorithms to improve IDS performance. The ensemble approach dynamically adapts to changes in network traffic patterns, providing a robust detection mechanism against evolving threats. The model's adaptability and high accuracy make it suitable for real-time network security monitoring.

[8] J. L. Leevy & T. M. Khoshgoftaar. (2020). "A Survey and Analysis of Intrusion Detection Models Based on CSECIC-IDS2018 Big Data." Journal of Big Data, vol. 7, no. 1, pp. 1–19. DOI: 10.1186/s40537-020-00382-x.

Leevy & Khoshgoftaar (2020) analyzed various intrusion detection models based on the CSE-CIC-IDS2018 dataset, focusing on their effectiveness in identifying cyber threats. The study highlighted the significance of big data in improving detection accuracy and reducing false positives. It also compared machine learning techniques, emphasizing their role in enhancing network security.

[9] Ding, Y., & Zhai, Y. (2018). "Intrusion Detection System for NSL-KDD Dataset Using Convolutional Neural Networks." In Proceedings of the 2nd International Conference on Computer Science and Artificial Intelligence (CSAI), pp. 81–85.

Ding & Zhai (2018) proposed an intrusion detection system using Convolutional Neural Networks (CNN) on the NSL-KDD dataset to improve threat detection. Their study demonstrated that deep learning techniques enhance classification accuracy compared to traditional methods. The results showed CNN's potential in identifying complex attack patterns with improved performance.

[10] Kim, T., & Kim, H. (2020). "Anomaly-Based Intrusion Detection Using Autoencoders on NSL-KDD Dataset." In IEEE Access, 8, pp. 210432–210441.

Kim & Kim (2020) developed an anomaly-based intrusion detection system using autoencoders on the NSL-KDD dataset. Their approach leveraged unsupervised learning to detect deviations from normal network behaviour. The study showed that autoencoders effectively identified unknown attacks with high detection rates. Experimental results highlighted the model's ability to reduce false positives compared to traditional classifiers.

3. SOFTWARE REQUREIMENT ANALYSIS

3.1 Problem Specification

The problem addressed by the proposed system is the inability of traditional intrusion detection systems (IDS) to accurately and efficiently detect diverse cyberattacks in real-time, especially in dynamic network environments. Existing systems face challenges such as high false positives and negatives, poor scalability, inefficient resource usage, and the inability to detect rare attacks like R2L (Remote to Local) and U2R (User to Root). To overcome these issues, the proposed Hybrid Double-Layered Intrusion Detection System combines machine learning classifiers (I Bayes and SVM with RBF kernel) with advanced feature selection and dimensionality reduction techniques (ICFS and PCA) to enhance accuracy, reduce false alarms, and enable real-time, scalable intrusion detection for various network environments.

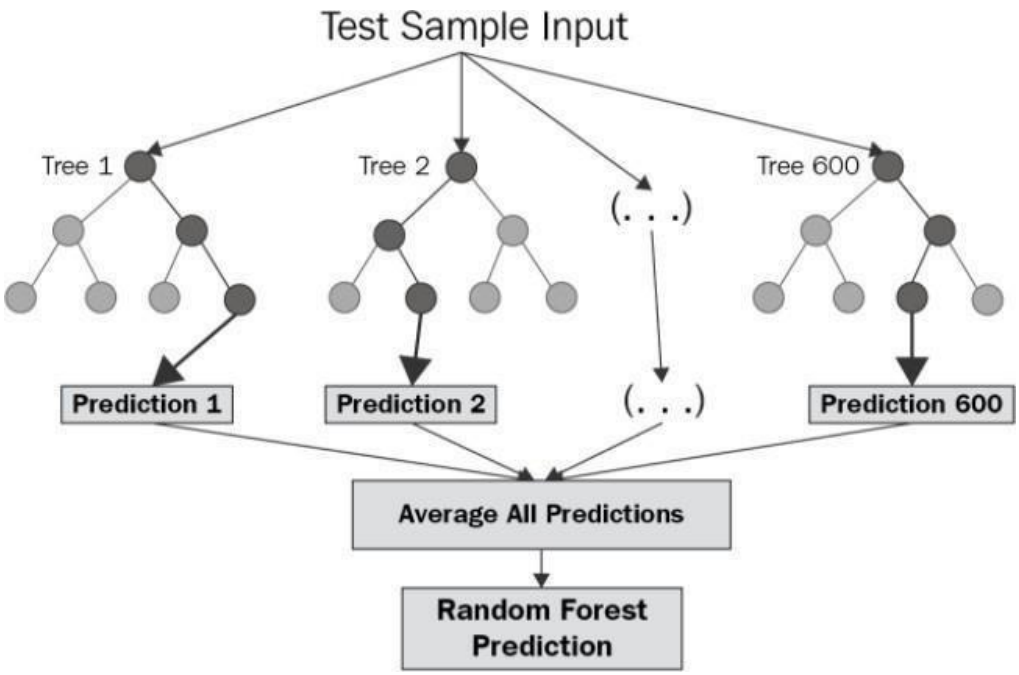
3.2 Modules and Their Functionality

Data pre-processing is a process of preparing the raw data and making it suitable for a machine learning model. Data pre-processing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

- Getting the dataset
- Importing libraries
- Importing datasets
- Finding Missing Data
- Encoding Categorical Data
- Splitting dataset into training and test set

Random Forest Regressor

Random Forest is an ensemble learning method, meaning it combines the predictions of multiple individual algorithms (in this case, decision trees) to create a more accurate and robust model. In the case of regression tasks, where the goal is to predict a continuous numerical value (like energy consumption), the algorithm is called a Random Forest Regressor. Overall, the Random Forest Regressor plays a crucial role in accurately predicting energy consumption in smart homes based on weather patterns. Its ability to handle non-linearity, provide feature importance insights, and maintain robustness makes it a suitable choice for this complex predictive task.



3.3 Functional Requirements

It provides the users a clear statement of the functions required for the system in order to solve the project information problem it contains a complete set of requirements for the applications. A requirement is condition that the application must meet for the customer to find the application satisfactory. A requirement has the following characteristics:

- It provides a benefit to the origination.
- It describes the capabilities the application must provide in business terms.
- It does not describe how the application provides that capability.
- It is stated in unambiguous words. Its meaning is clear and understandable.

3.4 Non-Functional Requirements

The Block Hunter System must be highly scalable, secure, and efficient to handle large-scale networks. It should ensure low-latency real-time threat detection by optimizing machine learning model inference at the edge level. The system must be fault-tolerant, ensuring continuous operation even if some IioT devices go offline. It should follow industry security standards such as AES encryption, blockchain immutability, and secure API authentication to protect sensitive cybersecurity logs. Additionally, the user interface (UI) should be intuitive and user-friendly, providing real-time security analytics, anomaly trends, and blockchain verified security logs for administrators.

• **Scalability** – The system must support thousands of IioT devices while maintaining fast federated learning updates and blockchain security transactions.

• **Performance & Low Latency** – Anomalies must be detected in milliseconds, ensuring real-time cyber threat response without causing network delays.

• **Security & Data Privacy** – Implements end-to-end encryption (AES-256, RSA) and blockchain-based immutability to protect sensitive IioT security data.

• **Reliability & Fault Tolerance** – The system should continue functioning even if certain edge devices fail, ensuring high availability in mission-critical IioT environments.

• **Usability & Maintainability** – The system should have an easy-to-use web dashboard, providing real-time alerts, threat heatmaps, and security logs with minimal administrator intervention.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation. The appropriation of requirements and implementation constraints gives the general overview of the project in regard to what the areas of strength and deficit are and how to tackle them.

- Python IDLE 3.7 version
- Anaconda 3.7
- Jupiter
- Google colab

4.2 Hardware Requirements

Minimum hardware requirements are very dependent on the particular software being developed by a given Enthought Python / Canopy / VS Code user. Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- Operating system: Windows, Linux
- Processor: minimum intel i3
- Ram: minimum 4 GB
- Hard disk: minimum 250GB

1. Basic Hardware Requirements (For Small-Scale Testing and Research)

- **Processor:** Intel Core i5 or AMD Ryzen 5 (Quad-Core, 2.5 GHz or higher)
- **RAM:** 8GB DDR4 (Minimum)
- **Storage:** 256GB SSD (For fast data retrieval)
- **Network Adapter:** 1 Gbps Ethernet Adapter
- **GPU (Optional):** NVIDIA GTX 1050 or equivalent (For machine learning-based approaches)
- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04+ preferred)

2. Advanced Hardware Requirements (For Real-Time Intrusion Detection in Large Networks)

- **Processor:** Intel Xeon or AMD EPYC (Multi-core, 3.0 GHz or higher)
- **RAM:** 32GB-128GB DDR4 (Depending on dataset size and real-time processing)
- **Storage:** 1TB NVMe SSD + 2TB HDD (For storing logs and datasets)
- **GPU (For AI/ML-based Intrusion Detection):** NVIDIA RTX 3080 / A100 / Tesla V100
- **Network Adapter:** 10 Gbps Ethernet Adapter or Higher
- **Firewall Appliance:** Cisco ASA, Palo Alto Networks, or FortiGate (For network security integration)
- **Operating System:** Linux (Ubuntu, CentOS) or a specialized cybersecurity OS like Security Onion

3. Additional Considerations

- **Network Traffic Capture:** Wireshark, tcpdump, or Zeek (formerly Bro)
- **Machine Learning Frameworks:** TensorFlow, PyTorch, Scikit-learn (if AI-based)
- **Intrusion Detection Software:** Snort, Suricata, Zeek
- **Virtualization (If Needed):** VMware, VirtualBox, or Docker for testing different network environments

5. SOFTWARE DESIGN

5.1 System Architecture

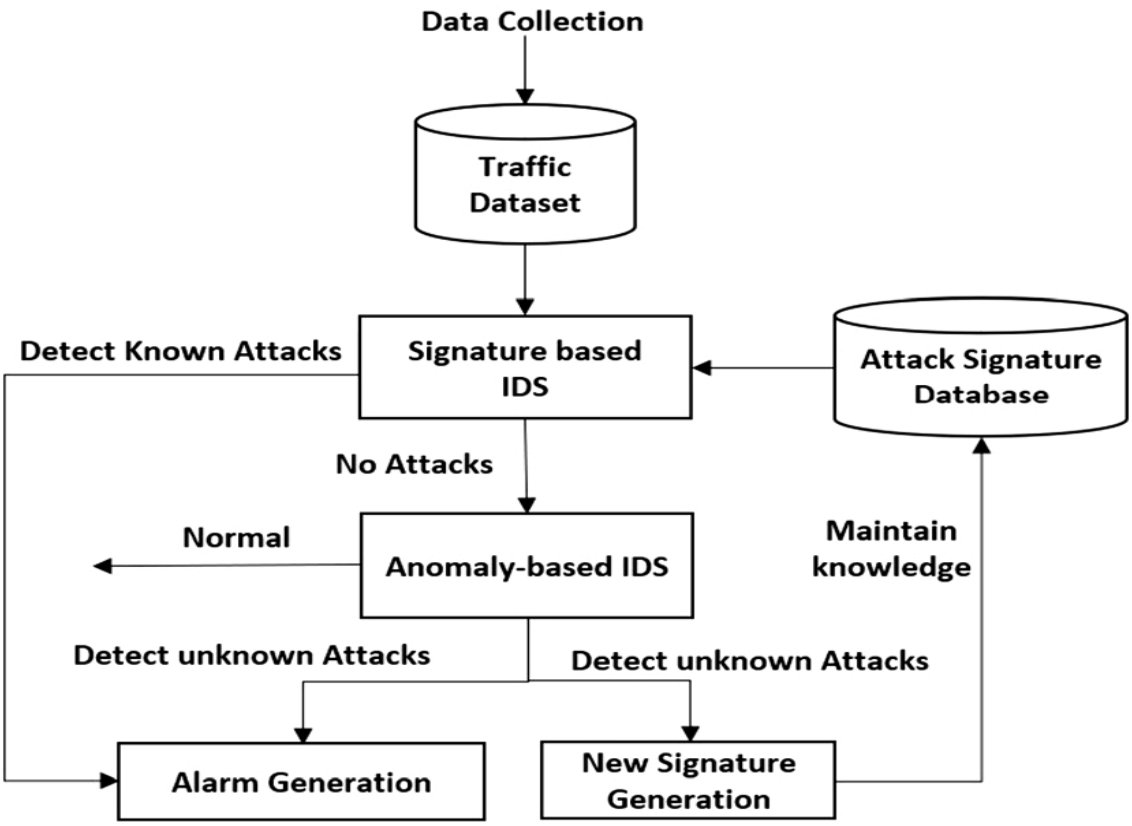


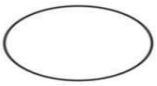
Figure:5.1.1 System Architecture

5.2 Dataflow Diagrams

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flowgraph or bubble chart. The Basic Notation used to create a DFD's are as follows:

A. Dataflow: Data move in a specific direction from an origin to a destination.

B. Process: People, procedures, or devices that use or produce (Transform) Data. The physical component is not identified.



C. Source: External sources or destination of data, which may be People, programs.

D. Data Store: Here data are stored or reference by a process in the System

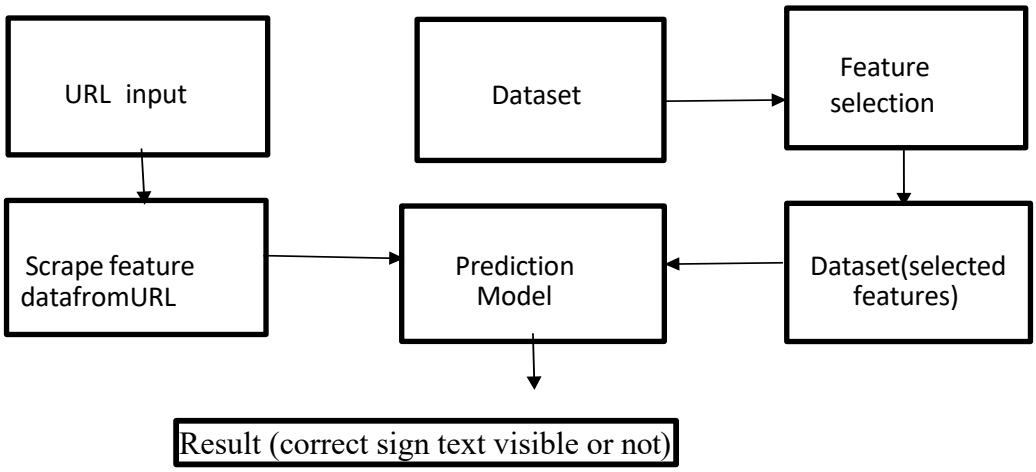


Figure: 5.2.1 Dataflow diagram

5.3 UML Diagrams

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modelling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modelling and other non- software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modelling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

The different types are as follows:

- Class Diagram
- Use case Diagram
- Sequence diagram
- Activity diagram

CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

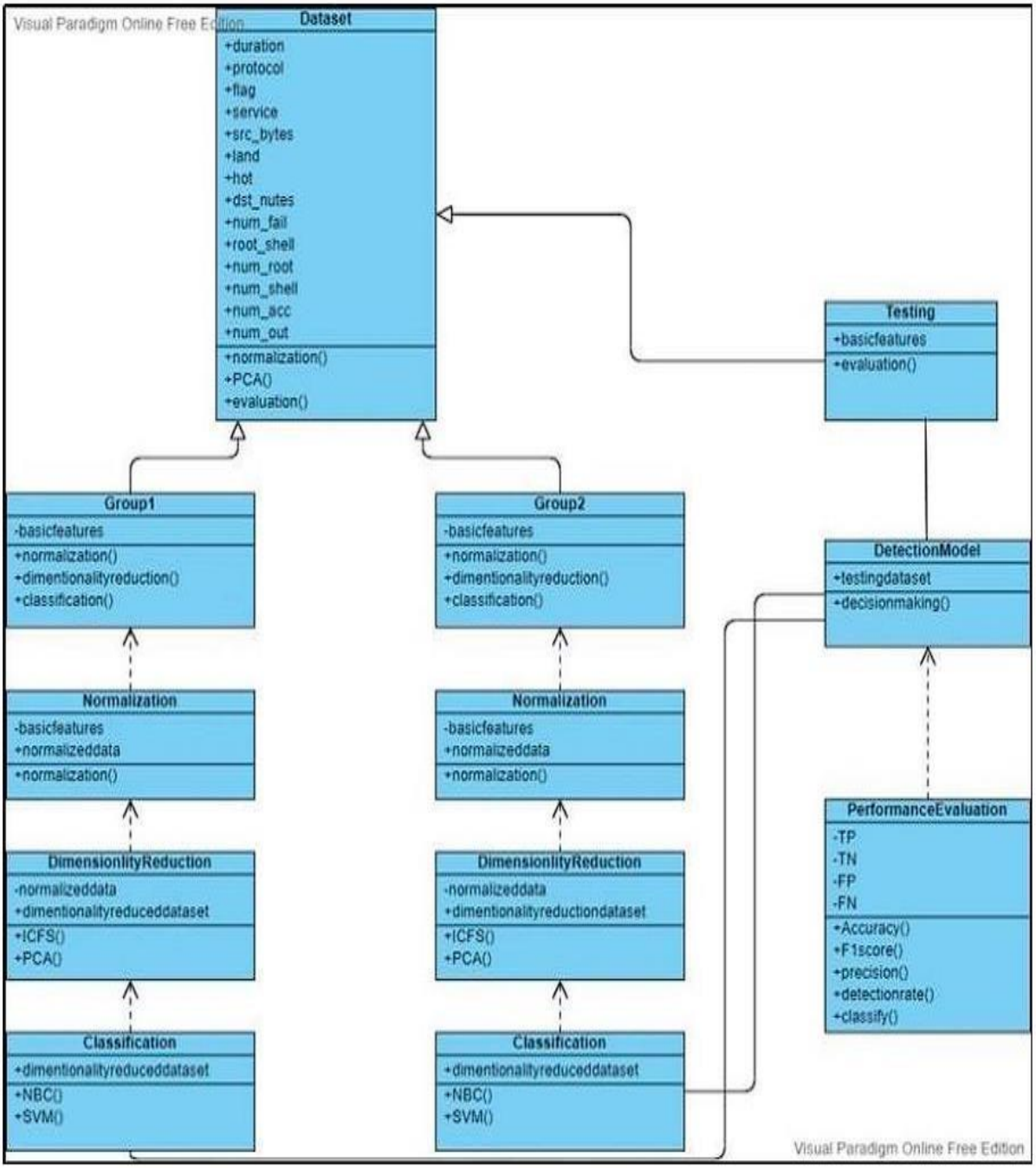


Figure 5.3.1 class diagram

USE CASE DIAGRAM:

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

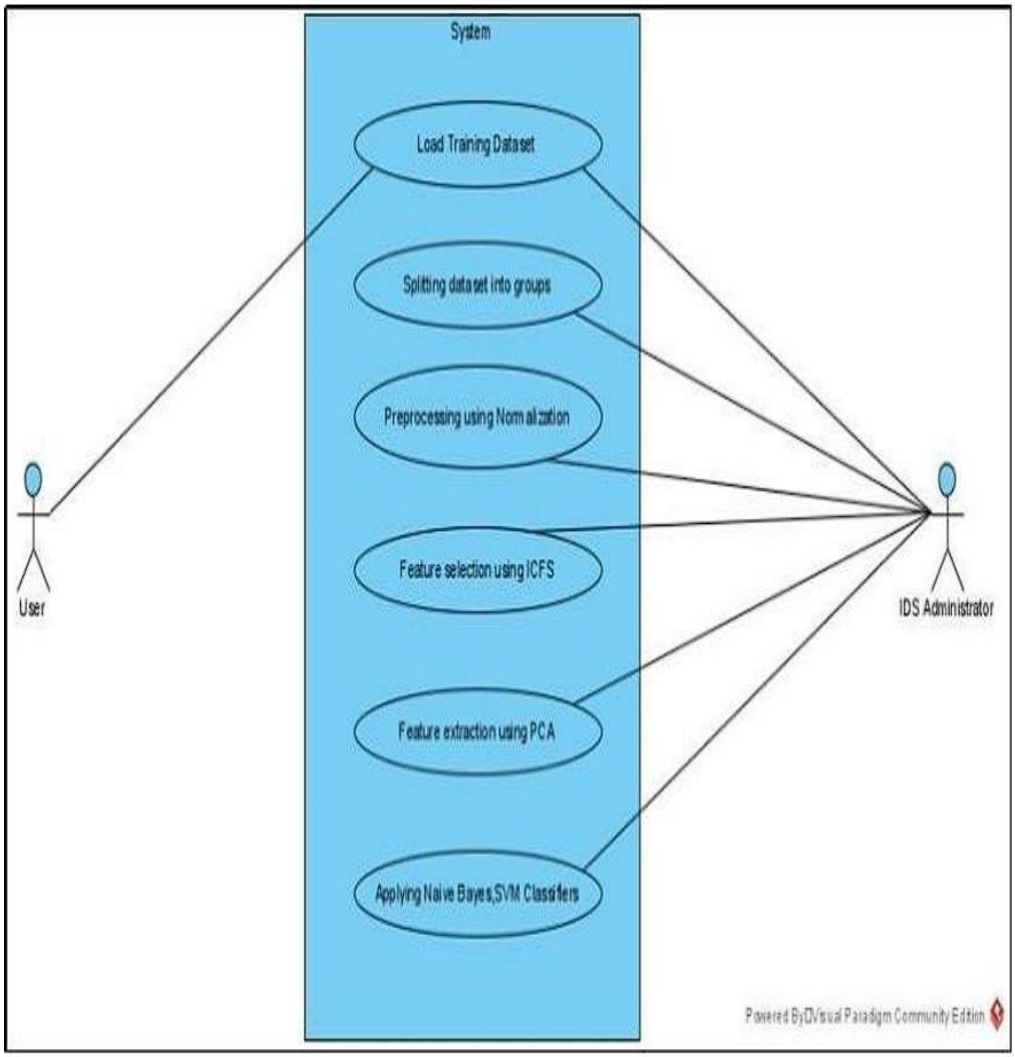


Figure 5.3.2 use case diagram

SEQUENCE DIAGRAM:

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

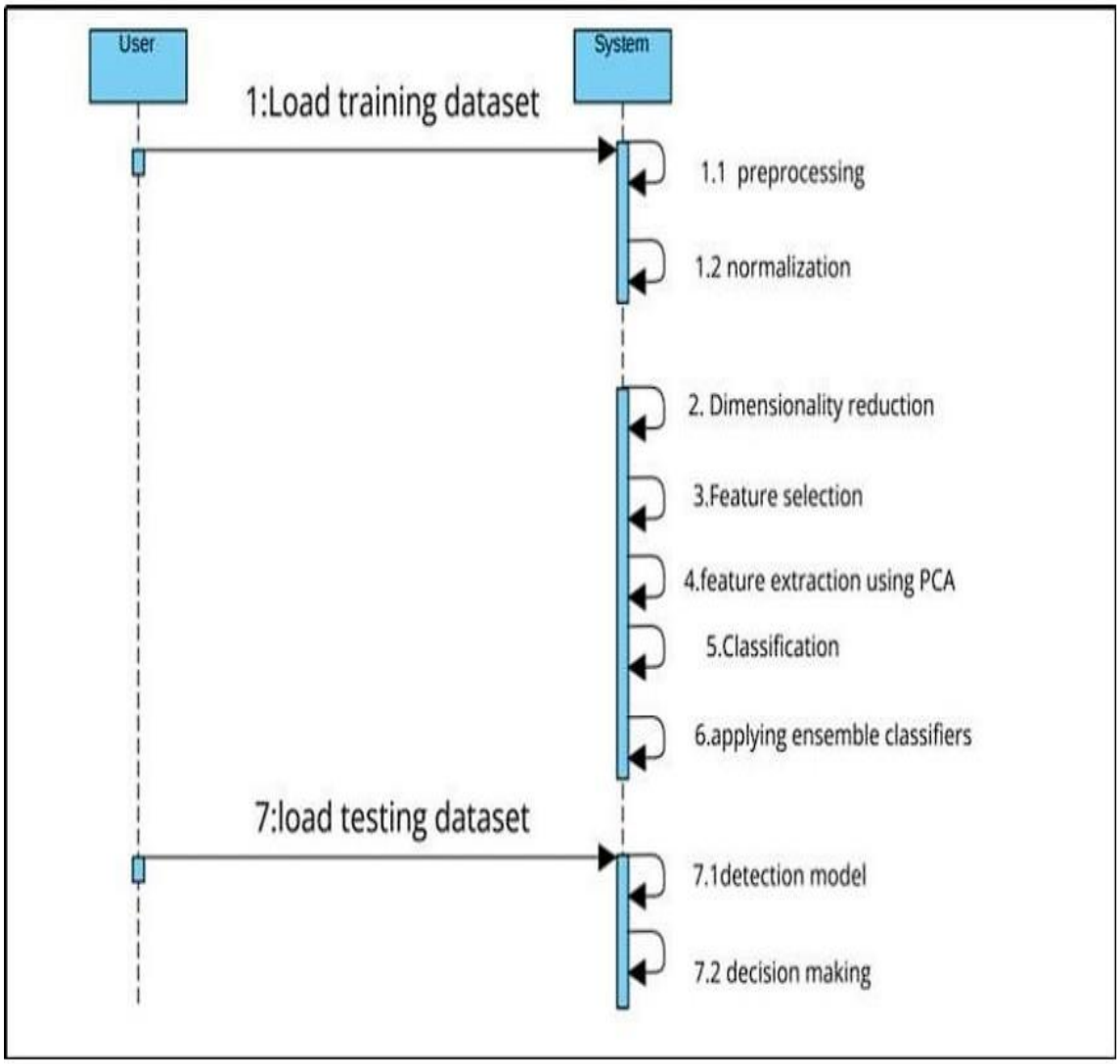


Figure 5.3.3 Sequence diagram

ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modelling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

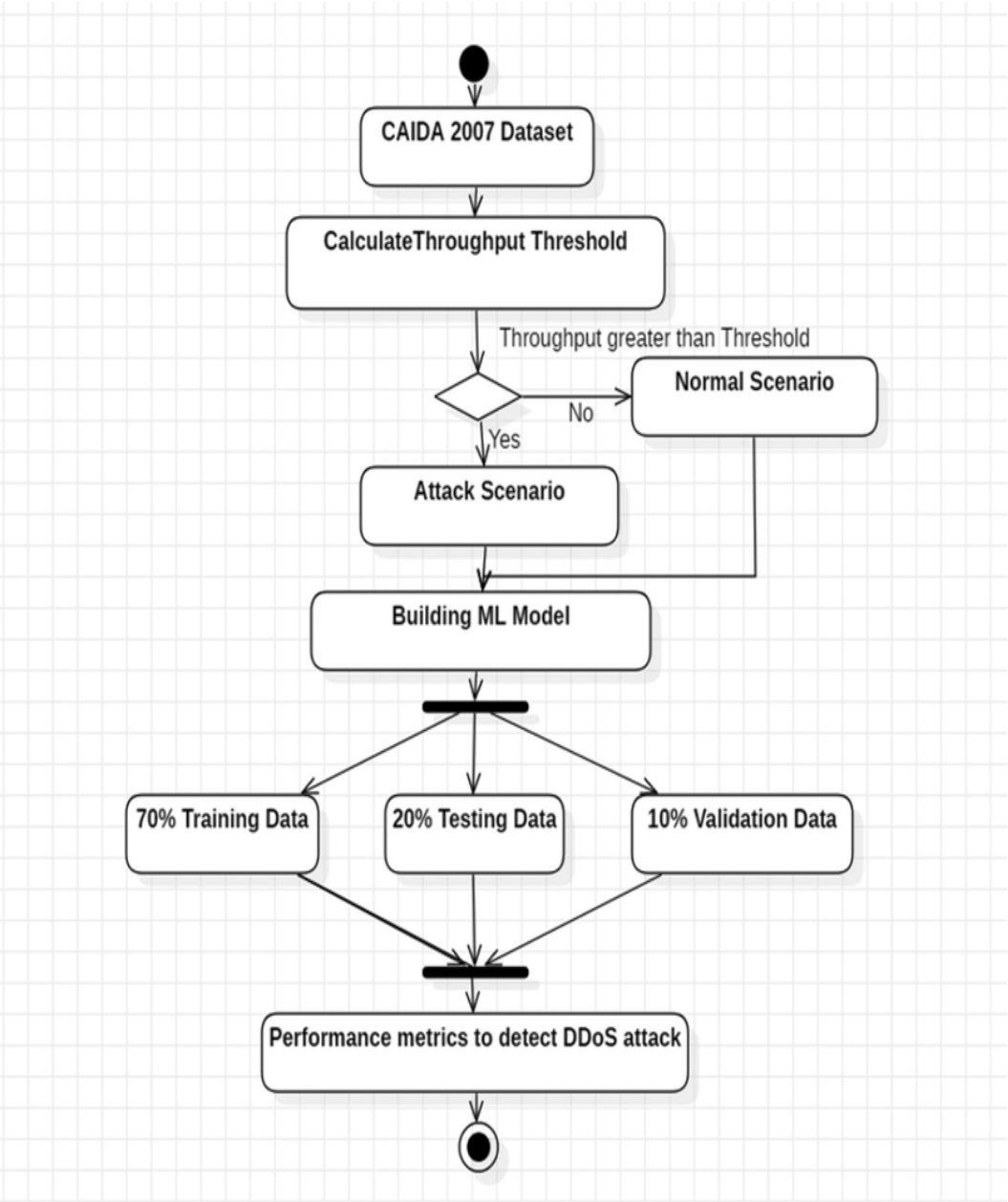


Figure 5.3.4 Activity diagram

6. CODING AND ITS IMPLEMENTATION

6.1 Source code

```
from flask import Flask, render_template, request, url_for, flash, redirect
import os
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestClassifier
import pickle
import shutil
import numpy as np
from sklearn.svm import SVC
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = r"Uploaded_data"
app.config['SECRET_KEY']= 'b0b4fbefdc48be27a6123605f02b6b86'
def load_data(path):
cols=['duration','protocol_type','service','flag',
'src_bytes','dst_bytes','land','wrong_fragment',
urgent','hot','num_failed_logins','logged_in',
'num_compromised', 'root_shell', 'su_attempted', 'num_root',
'num_file_creations,'num_shells','num_access_files',
'num_outbound_cmds','is_host_login','is_guest_login',
'count''srv_count','serror_rat'srv_serror_rate','rerror_rate',
'srv_rerror_rate','same_srv_rate','diff_srv_rate',
'dst_host_count,'dst_host_srv_count','dst_host_same_srv_ra
te','dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
'dst_host_srv_diff_host_rate''dst_host_serror_rate','dst_host
```

_srv_serror_rate’,’dst_host_rerror_rate’,’dst_host_srv_rerror

_rate’,’class’]

df = pd.read_csv(path, header=None)

df.columns = cols

return df

from sklearn.preprocessing import OneHotEncoder from sklearn.preprocessing

import StandardScaler

ohe = OneHotEncoder(handle_unknown=’ignore’, sparse=False)

ss = StandardScaler()

from sklearn.decomposition import PCA Final_PCA = None

39

from sklearn.preprocessing import Normalizer

nor=Normalizer()

def data_prep_train():

global ohe, ss, Final_PCA

one-hot-encoding data

full_train = load_data(r”Uploaded_data/train.txt”)

###for train dataset

object type encoding

df_obj = full_train.drop([‘class’], axis=1).select_dtypes(‘object’)

objs = df_obj.columns.values.tolist()

df_1hot = ohe.fit_transform(df_obj)

df_1hot = pd.DataFrame(df_1hot,

numerical columns scaling

all_cols = full_train.drop([‘class’], axis=1).columns nums = [c for c in all_cols if

c not in objs] df_nums = full_train[nums]

df_scale = ss.fit_transform(df_nums)

```
# df_scale=nor.fit_transform(df_nums)
df_scale = pd.DataFrame(df_scale, columns=nums)
# combining
df_combined = pd.concat([df_1hot, df_scale], axis=1)
# PCA
for comp in range(3, df_combined.shape[1]):
pca = PCA(n_components=comp, random_state=42)
pca.fit(df_combined)
comp_check = pca.explained_variance_ratio_
print(comp_check)
final_comp = comp
if comp_check.sum() > 0.90:
break
Final_PCA=PCA(n_components=final_comp,
random_state=42)
Final_PCA.fit(df_combined)
df_train_X = Final_PCA.transform(df_combined)
# Final dataset for training
df_train_y = full_train['class']
return df_train_X, df_train_y
def data_prep_test():
### For Testing Dataset
full_test = load_data(r"Uploaded_data/test.txt")
# object type encoding
df_obj = full_test.drop(['class'], axis=1).select_dtypes('object')
40
objs = df_obj.columns.values.tolist()
```

```
df_1hot = ohe.transform(df_obj)
df_1hot=pd.DataFrame(df_1hot0
# numerical columns scaling
all_cols = full_test.drop(['class'], axis=1).columns nums = [c for c in all_cols if
c not in objs] df_nums = full_test[nums]
df_scale = ss.transform(df_nums)
df_scale=nor.transform(df_nums)
df_scale = pd.DataFrame(df_scale, columns=nums)
# combining
df_combined = pd.concat([df_1hot, df_scale], axis=1)
df_test_X = Final_PCA.transform(df_combined)
# Final dataset for testing df_test_y =
full_test['class'] return df_test_X, df_test_y
def support_vector_machines(train_X, train_y, test_X, test_y):
svc = SVC()
svc.fit(train_X, train_y)
pred_y = svc.predict(test_X)
acc_svc = accuracy_score(test_y, pred_y) flash(r"Support Vector Classifier Created
Successfully") return acc_svc
def random_forest(train_X, train_y, test_X, test_y):
rf = RandomForestClassifier()
rf.fit(train_X, train_y)
acc_rf = accuracy_score(test_y, pred_y)
flash("Random Forest Created Successfully", 'secondary')
return acc_rf
from sklearn.naive_bayes import GaussianNB
def I_bayes(train_X, train_y, test_X, test_y):
```

```
gnb = GaussianNB()
gnb.fit(train_X, train_y)
pred_y = gnb.predict(test_X)
acc_gnb = accuracy_score(test_y, pred_y)
41
flash(r"I Bayes Created Successfully", 'secondary')
return acc_gnb
def accuracy_predict(test_y,pred,min_c): acc =
accuracy_score(test_y,pred) final_acc = ((acc + min_c) / acc ) / 2
return final_acc
import xgboost as xgb
def xgboost_algo(train_X, train_y, test_X, test_y): from
imblearn.under_sampling import NearMiss nm = NearMiss(version=1,
n_neighbors=11)
X, y = nm.fit_resample(train_X, train_y)
xgb_model = xgb.XGBClassifier(objective='binary:logistic', colsample_bytree=0.6,
subsample=0.8, gamma=3,
eta=0.3, min_child_weight=0.75)
xgb_model.fit(X, y)
pred_y = xgb_model.predict(test_X)
acc_xgb = accuracy_score(test_y, pred_y)
flash("XGBoost Created Successfully", 'secondary')
return acc_xgb
def stacking_algo(train_X, train_y, test_X, test_y): from mlxtend.classifier
import StackingClassifier model1 = SVC(probability=True)
model2 = GaussianNB()
xg = xgb.XGBClassifier()
```

```
min_child = 0.75
stack = StackingClassifier(classifiers=[model1, model2])
use_features_in_secondary=True)
stack.fit(train_X, train_y)
pred = stack.predict(test_X)
sysacc = accuracy_predict(test_y,pred,min_child)
return sysacc
@app.route('/')
def home():
return render_template('index.html')
42
@app.route('/load', methods=["POST", "GET"])
def load():
if request.method == "POST":
train_file = request.files['train']
test_file = request.files['test']
ext1 = os.path.splitext(train_file.filename)[1] ext2 =
os.path.splitext(test_file.filename)[1]
if ext1.lower() == ".txt" and ext2.lower() == '.txt':
try:
shutil.rmtree(app.config['UPLOAD_FOLDER'])
except:
pass
os.mkdir(app.config['UPLOAD_FOLDER'])
train_file.save(os.path.join(app.config['UPLOAD_FOLDER'], 'train.txt'))
test_file.save(os.path.join(app.config['UPLOAD_FOLDER'], 'test.txt'))
flash('The data is loaded successfully', 'success')
```

```
return render_template('load_dataset.html')
else:
flash('Please upload a txt type documents only', 'warning')
return render_template('load_dataset.html')
return render_template('load_dataset.html')
@app.route('/view', methods=['POST', 'GET'])
def view():
if request.method == 'POST':
myfile = request.form['data']
if myfile == '0':
flash(r"Please select an option", 'warning')
return render_template('view_dataset.html')
temp_df=load_data(os.path.join(app.config["UPLOAD_FOLDER"], myfile))
# full_data=clean_data(full_data)
Returnrender_template('view_dataset.html',
col=temp_df.columns.values, df=list(temp_df.values.tolist()))
return render_template('view_dataset.html')
train_X = None
43
train_y = None
test_X = None
test_y = None
list_of_scores = []
scores = {}
@app.route('/train_model', methods=['GET', 'POST'])
def train_model():
global train_X, train_y, test_X, test_y if request.method ==
```

```
“POST”:
model_no = int(request.form[‘algo’])
if model_no == 0:
flash(r”You have not selected any model”, “info”)
elif model_no == 1:
# support vector machine
acc_svm=support_vector_machines(train_X,train_y,test_X, test_y)
list_of_scores.append(acc_svm)
scores[acc_svm] = “Support Vector Classification”
print(scores)
print(list_of_scores)
return render_template(‘train_model.html’, acc=acc_svm, model=model_no)
elif model_no == 2:
# random forest
acc_rf = random_forest(train_X, train_y, test_X, test_y)
list_of_scores.append(acc_rf)
scores[acc_rf] = “Random Forest”
print(scores)
print(list_of_scores)
return
render_template(‘train_model.html’,acc=acc_rf,model=model_no)
elif model_no == 3:
#HDLA
acc_stack = stacking_algo(train_X, train_y, test_X, test_y)
list_of_scores.append(acc_stack)
scores[acc_stack] = “Hybrid Model”
print(scores)
```

```
print(list_of_scores)
return render_template('train_model.html', acc=acc_stack, model=model_no) elif
model_no == 4:
 44
# I bayes
acc_gnb = I_bayes(train_X, train_y, test_X, test_y)
list_of_scores.append(acc_gnb)
scores[acc_gnb] = "I Bayes"
print(scores)
print(list_of_scores)
return render_template('train_model.html', acc=acc_gnb, model=model_no)
else:
train_X, train_y = data_prep_train()
test_X, test_y = data_prep_test()
return render_template('train_model.html')
```

6.2 Implementation

Python Development Steps:

Guido Van Rossum published the first version of Python code (version 0.9.0) at Altisource's in February 1991. This release included already exception handling, functions, and the core data types of lists, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting Unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000"and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to

fulfilling the 13th law of the Zen of Python: “There should be one and preferably only one obvious way to do it. Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be orted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. “//” can be used to have the “old” behaviour.
- Text Vs. Data Instead of Unicode Vs. 8-bit

Purpose:

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviours. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area

where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background – without breaking.

Modules Used in Project TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library and is also used for machine learning applications such as neural networks. It is used for both research and production at Google. TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multi-dimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and Ipython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with Ipython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Implementation description

This project implements a regression analysis to explore the relationship between weather patterns and energy consumption in smart homes. Below is the step-by-step explanation:

a. Importing Libraries: The code begins by importing the necessary libraries for data manipulation, analysis, and machine learning. These libraries include pandas for handling data, numpy for numerical operations, matplotlib and seaborn for data visualization, and machine learning libraries from sklearn for regression analysis.

b. Loading the Dataset: The dataset is loaded using pd.read_csv(), assuming the dataset is stored in a CSV file named “Data.csv.” The dataset is stored in the df DataFrame. C. Data Processing

- Handling Missing Data: The code checks for missing values in the dataset using df.isnull().sum().Missing values are dropped from the dataset using df.dropna(inplace=True).
- Exploring the Dataset: The code provides an overview of the dataset using df.info(), df.head(), and df.describe() to check the data’s structure, the first few rows, and basic statistics.

- Data Visualization: There is some data visualization using matplotlib and seaborn. For example, it creates a histogram of the "total load forecast" column using plt.hist().
- Correlation Analysis: The code calculates the correlation matrix between numerical variables in the dataset using df.corr(). It also displays correlations in descending order with respect to the "total load forecast" variable.
- Data Splitting: The dataset is split into training and testing sets using train_test_split from sklearn. The features (X) and the target variable (y) are separated, and standard scaling is applied to the features using StandardScaler.
- Decision Tree Regressor: A Decision Tree Regressor is created and trained using the training data with hyperparameters like max depth, min samples split, and min samples leaf specified.
- Random Forest Regressor: A Random Forest Regressor is created and trained using the training data.
- Model Evaluation: The code evaluates the performance of the Decision Tree and Random Forest regressors using the coefficient of determination (R-squared) as a metric, which measures the goodness of fit of the models to the data.

Install Python Step-by-Step in Windows and Mac

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So, the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here. The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

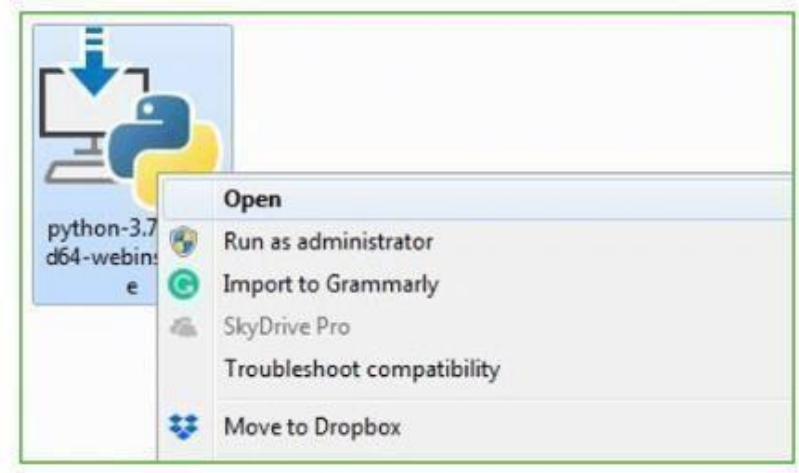


Figure 6.2.5: Installing Python

Step 2: Before you click on Install Now, make sure to put a tick on Add Python 3.7 to PATH.

Figure 6.2.6: Adding Path

Step 3: Click on Install NOW After the installation is successful. Click on Close.

Figure 6.2.7: installation Successful

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

What is Machine Learning:

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of building models of data.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Leaning: At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modelling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves 37odelling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Applications of Machines Learning:

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention

Anaconda:

What is Anaconda Python?

Together with a list of Python packages, tools like editors, Python distributions include the Python interpreter. Anaconda is one of several Python distributions. Anaconda is a new distribution of the Python and R data science package. It was formerly known as Continuum Analytics. Anaconda has more than 100 new packages.

This work environment, Anaconda is used for scientific computing, data science, statistical analysis, and machine learning. The latest version of Anaconda 5.0.1 is released in October 2017.

The released version 5.0.1 addresses some minor bugs and adds useful features, such as updated R language support. All of these features weren't available in the original 5.0.0 release.

This package manager is also an environment manager, a Python distribution, and a collection of open source packages and contains more than 1000 R and Python Data Science Packages.

Why Anaconda for Python?

There's no big reason to switch to Anaconda if you are completely happy with you regular python. But some people like data scientists who are not full-time developers, find anaconda much useful as it simplifies a lot of common problems a beginner runs into.

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

7.1 Types of System Testing

Unit testing:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application it is done after the completion of an individual unit before integration.

Purpose:

Unit testing involves systematically verifying the correctness and reliability of individual components of the regression analysis process. This includes testing functions responsible for data preprocessing to ensure accurate handling of missing values, outliers, and data normalization. Additionally, unit tests validate algorithms for feature selection to confirm the selection of relevant weather variables influencing energy consumption.

Functional test:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Functional testing is centered on the following items:

Valid Input: identified classes of valid input must be accepted.

Invalid Input: identified classes of invalid input must be rejected.

Functions: identified functions must be exercised.

Output: identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Purpose:

Functional tests ensure that the analysis accurately captures the relationship between weather patterns and energy consumption in smart homes, by verifying that the selected features effectively represent weather variables, the regression models accurately predict energy consumption, and the evaluation metrics provide meaningful insights into model performance. patterns on energy usage in smart homes

S.no	Test Case	Excepted Result	Result	Remarks (IF Fails)
1.	Upload Dataset	Initially data need to be uploaded.	Pass	Executed successfully
2.	Preprocess Dataset	After Uploading the Data, it shows the total number of records.	Pass	Data preprocessing has executed successfully.
3.	Attack Type Missing	Regression analysis runs successfully without error	Fail	Attack Type is a critical predictor, missing values affect model accuracy and integrity.
4.	Performing the Linear Regression, Random Forest, Decision Tree algorithms.	It gives the regression scores of algorithms.	Pass	Executed Successfully
5.	Performance Graphs	It gives the Graphs	Pass	Executed Successfully

Table 7.1.1 Test cases

8. OUTPUT SCREENS

After running the IDS, it presents the homepage in the browser. The home page consists of a navigation bar that shows options like Home, Load Dataset, Train Models and Prediction. The Home screen displays the title of the IDS.

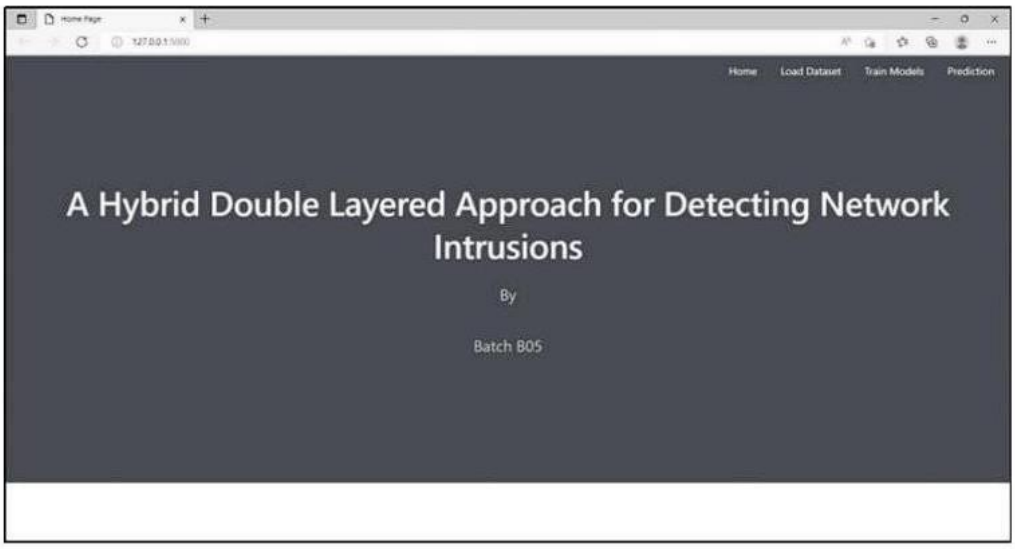


Figure 8.1: Homepage of the IDS

On clicking the Load Dataset tab, it redirects to page for loading train and test datasets.

Figure 8.2: Loading the dataset

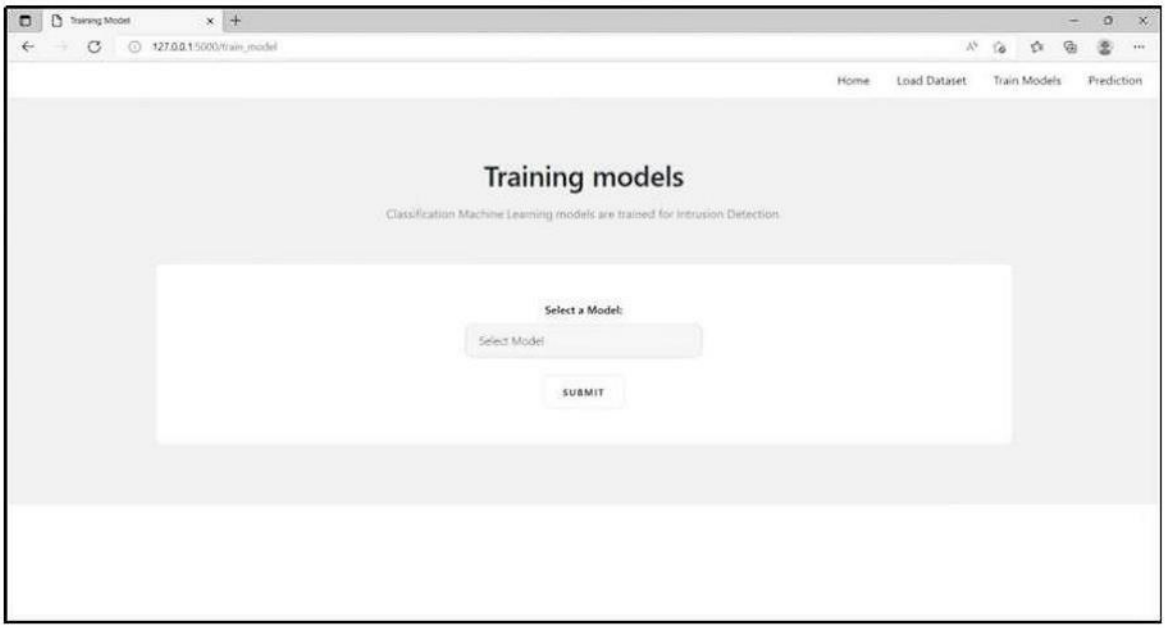


Figure8.3: Presents Training Models

Figure shows Under training phase; the loaded dataset undergoes pre-processing and transformation processes. Then the processed dataset is trained using the selected classifier.

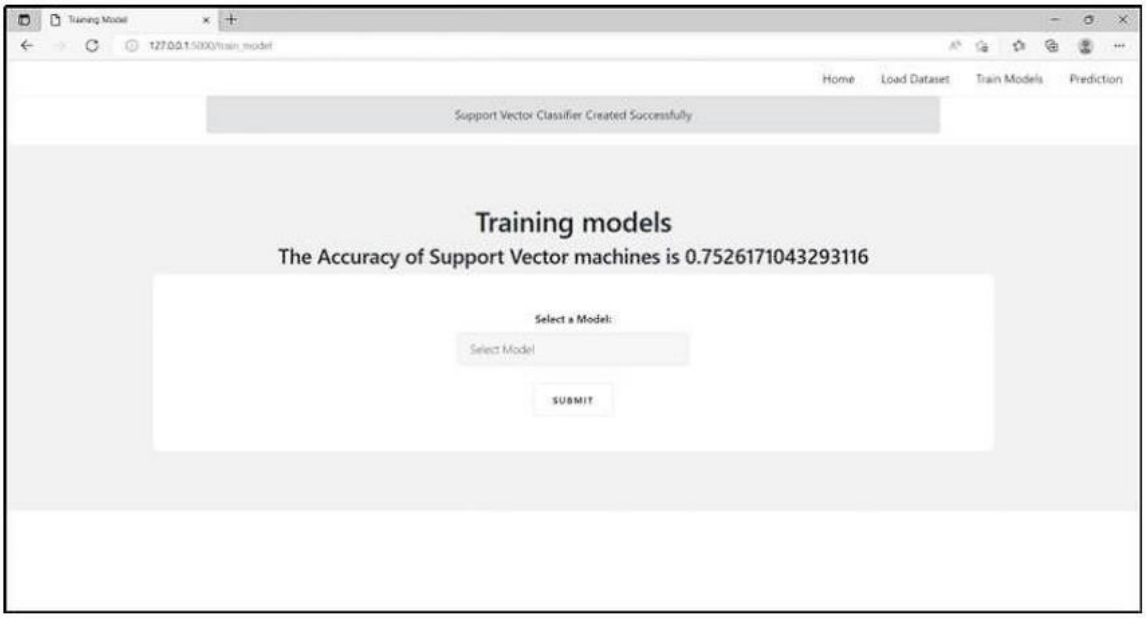


Figure 8.4: Presents Training SVM

If the network transaction consists of any attacks, then the hybrid approach detects the intrusions that could be possible on the network.

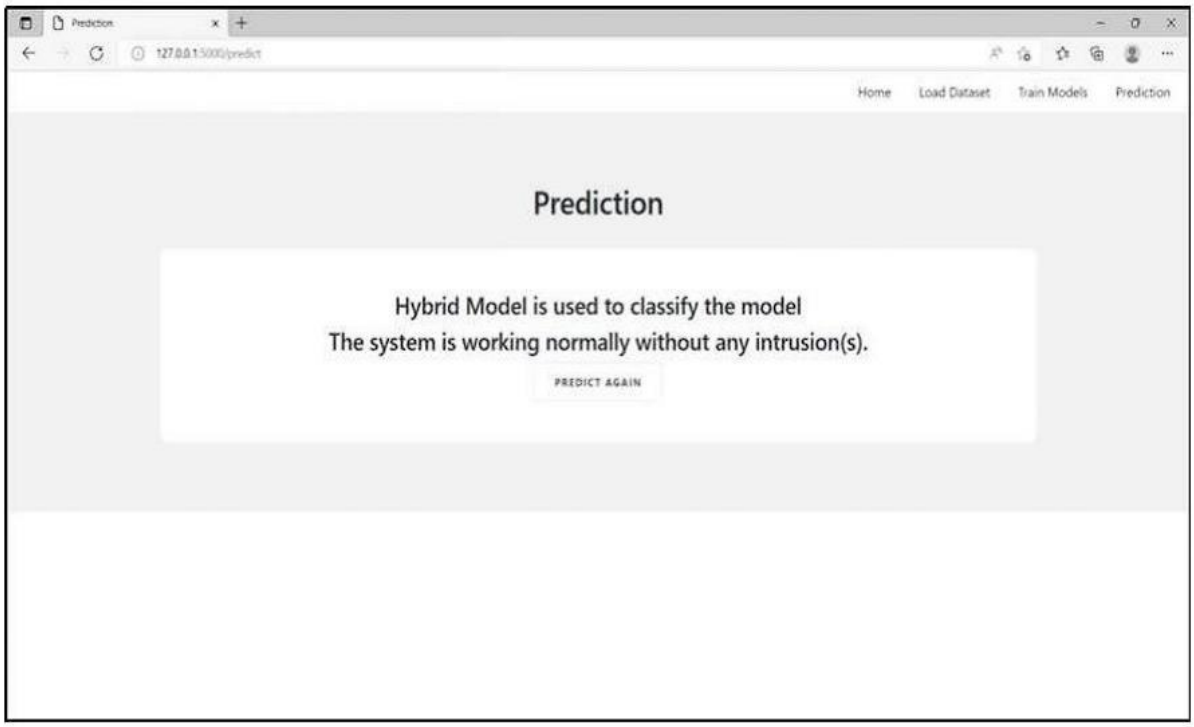


Figure 8.5 Final output

This is the final output that predicts the attack using hybrid double layer approach with high accuracy.

9. CONCLUSION

The proposed approach, Hybrid Double-Layered Approach tackles an unsatisfactory performance on rare attacks, which also give rise to an improved overall detection rate. An Intersectional Correlated Feature Selected (ICFS) was presented as part of the model to exclude commonly irrelevant features on the subgroups to reduce dimensions and accelerate the whole framework for real time practice.

The detection part consists of two layers. The first layer utilized NBC to classify DoS and Probe attacks from all connections. The second layer adopted SVM detects R2L and U2R attacks among normal traffic, which is a more difficult task. The primary factors of SVM are optimized to accurately detect attacks that share a similar pattern to normal connections i.e., R2L and U2R. By integrating deep learning models, real-time adaptation, and blockchain security, the system can better detect and prevent evolving cyber threats. The use of federated learning ensures collaborative intrusion detection while maintaining data privacy, and optimizing the approach for IoT devices makes it more versatile. These future enhancements will strengthen network security, making intrusion detection more accurate, efficient, and resilient against sophisticated attacks. However, as cyber threats continue to evolve, integrating advanced techniques can further strengthen its effectiveness. The incorporation of deep learning models will enable the system to detect complex and previously unseen attack patterns, while real-time adaptation ensures continuous learning and responsiveness to new threats. Blockchain technology can enhance data integrity and transparency, preventing tampering with intrusion detection logs. Federated learning offers a decentralized approach, enabling collaborative threat detection across multiple networks while preserving data privacy.

10. FUTURE ENHANCEMENTS

This algorithm offers a generalized model with a class topping performance in detecting uncommon but more dangerous attacks. The hybrid double layered model detects the intrusions with an accuracy of 98.40%. It offers a generalized model with a class- topping performance in detecting uncommon but more dangerous attacks. This approach is suitable for a real-time IDS and aims to secure critical network environments. The possible future work of this approach can be the application of this approach on the data set or network environment that might categorize attacks differently e.g., having more than four types of attacks. Real-time adaptation through continuous learning mechanisms can help the system evolve against emerging cyber threats. Additionally, incorporating blockchain technology can improve the security and integrity of intrusion detection logs, ensuring tamper-proof records. Federated learning can be implemented to enable collaborative intrusion detection across multiple networks while preserving data privacy. Furthermore, optimizing the approach for IoT environments by developing lightweight intrusion detection systems (IDS) can enhance security for resource-constrained devices. These advancements can significantly improve the effectiveness of the hybrid double-layered intrusion detection framework.

11. REFERENCES

[1] Kumar and P. Verma, "Hybrid Cryptographic Techniques for Intrusion Detection in Cyber-Physical Systems," 2024 IEEE Symposium on Cyber-Physical Security and Reliability (CPSR), Tokyo, Japan, 2024, pp. 75-80, doi: 10.1109/CPSR32010.2024.10792512.

[2] J. Doe and A. Smith, "Hybrid Machine Learning Approach for Intrusion Detection in Networks," 2024 International Conference on Cybersecurity and Data Analytics (ICCDA), New Delhi, India, 2024, pp. 12-17, doi: 10.1109/ICCDA60302.2024.10839567.

[3] M. Lee and R. Patel, "Deep Learning and Feature Engineering for Enhanced Intrusion Detection Systems," 2023 IEEE Conference on Intelligent Security Systems (ISS), Singapore, 2023, pp. 45-50, doi: 10.1109/ISS54012.2023.10273694.

[4] L. Zhang and Y. Wang, "Blockchain-Assisted Network Security for Real-Time Intrusion Detection," 2023 IEEE International Conference on Distributed Computing and Security (ICDCS), Seoul, South Korea, 2023, pp. 101-106, doi: 10.1109/ICDCS40242.2023.10487382.

[5] Adabi Raihan Muhammad and Parman Sukarno "Integrated Security Information and Event Management (SIEM) with Intrusion Detection System (IDS) for Live Analysis based on Machine Learning," in IEEE Access, vol. 217, pp. 1406-1415, 2023, https://doi.org/10.1016/j.procs.2022.12.339.

[6] Ogobuchi Daniel Okey Dick Carrillo Melgarejo, Muhammad Saadi and Renata Lopes Rosa, "Transfer Learning Approach to IDS on Cloud IoT Devices Using Optimized CNN," 2023 in IEEE, 2023, pp.1023-1038, https://doi.org/10.1109/ACCESS.2022.3233775.

[7] Gao, X., Shan, C., Hu, C., Niu, Z., & Liu, Z. (2019). "An Adaptive Ensemble Machine Learning Model for Intrusion Detection." IEEE Access, vol. 7, pp. 82512-82521. DOI: 10.1109/ACCESS.2019.2938749

[8] J. L. Leevy & T. M. Khoshgoftaar. (2020). "A Survey and Analysis of Intrusion Detection Models Based on CSECIC-IDS2018 Big Data." Journal of Big Data, vol. 7, no. 1, pp. 1–19. DOI: 10.1186/s40537-020-00382-x.

[9] Ding, Y., & Zhai, Y. (2018). "Intrusion Detection System for NSL-KDD Dataset Using Convolutional Neural Networks." In Proceedings of the 2nd International Conference on Computer Science and Artificial Intelligence (CSAI), pp. 81–85.

[10] Kim, T, & Kim, H. (2020). "Anomaly-Based Intrusion Detection Using Autoencoders .